



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

COMPUTAÇÃO DE ALTO DESEMPENHO

DOCENTE: CARLA DOS SANTOS SANTANA

E SAMUEL XAVIER DE SOUZA

DISCENTE: CLARA VIRGÍNIA NASCIMENTO SANTOS (20240002212)

Tarefa 8

1. ENUNCIADO

Implemente a estimativa estocástica de π usando `rand()` para gerar os pontos. Cada thread deve usar uma variável privada para contar os acertos e acumular o total em uma variável global com `#pragma omp critical`. Depois, implemente uma segunda versão em que cada thread escreve seus acertos em uma posição distinta de um vetor compartilhado. A acumulação deve ser feita em um laço serial após a região paralela. Compare o tempo de execução das duas versões. Em seguida, substitua `rand()` por `rand_r()` em ambas e compare novamente. Explique o comportamento dos quatro programas com base na coerência de cache e nos efeitos do falso compartilhamento.

2. DISCUSSÃO

2.1. Método Estocástico

A estimativa estocástica de π é um método que utiliza a geração de números aleatórios para estimar o valor de π . Pontos são gerados aleatoriamente dentro de um quadrado com um círculo inscrito; a proporção de pontos que caem dentro do círculo em relação ao total de pontos gerados é usada para estimar o valor de π convergindo para esta relação sendo $\pi/4$.

2.2. Race Condition

Ao paralelizar tarefas, um problema comum é a condição de corrida, quando múltiplas threads tentam acessar e modificar simultaneamente variáveis dependentes, o que pode trazer erros de registro, com valores errôneos que prejudicam o resultado final da tarefa. Para isso, existem diversas soluções para resolver essa problemática, como:

2.2.1. `#pragma omp critical`

Essa diretiva cria uma área dentro da região paralela que permite que apenas uma thread entre e execute o que está dentro do bloco critical, garantindo que a variável compartilhada seja modificada por apenas uma thread. Contudo, mesmo resolvendo o problema, essa diretiva não é a solução mais eficiente, já que as threads ficam esperando para acessar a região crítica, o que pode levar a um desempenho ruim.

2.2.2. `#pragma omp parallel + #pragma omp for`

Essas abordagem consegue separar a execução paralela da criação das tarefas em blocos iterativos, em que cada thread é responsável por um subconjunto dos pontos a serem gerados. Isso reduz a necessidade de sincronização já que não há dependência entre as iterações, pois cada thread trabalha com seus próprios dados locais. Após isso, cada thread fica esperando ao final para fazer a sincronização dos resultados (barreira implícita).

2.3. Estratégias de Acumulação dos Resultados

Ao final do laço paralelo, é necessário consolidar os acertos de cada thread em um único valor global. Seguem duas estratégias:

2.3.1. Variável privada + `#pragma omp critical`

Cada thread possui um contador local privado, incrementado livremente durante todo o laço sem qualquer sincronização. Apenas ao final, cada thread entra uma única vez em uma região critical para somar seu total ao acumulador global. Como o critical é executado poucas vezes (uma por thread), seu custo é desprezível, e o laço pesado opera inteiramente em cache privada.

2.3.2. Vetor compartilhado indexado por thread

Cria-se um vetor compartilhado em que cada thread escreve seus acertos na posição correspondente ao seu identificador. Como cada thread só acessa sua própria posição, não há condição de corrida lógica e o critical é dispensado. A acumulação final é feita por um laço serial após a região paralela. Contudo, essa abordagem pode sofrer com falso compartilhamento: as posições do vetor são logicamente independentes, mas residem fisicamente na mesma linha de cache, o que provoca invalidações constantes entre os núcleos.

2.4. Coerência de Cache e Falso Compartilhamento

Em sistemas multicore, cada núcleo possui sua própria cache, e a unidade mínima de transferência entre memória e cache é a linha de cache (geralmente 64 bytes). Quando uma thread escreve em uma posição de memória, o protocolo de coerência (MESI) invalida as cópias dessa linha presentes nas caches dos outros núcleos para manter consistência.

O falso compartilhamento ocorre quando threads diferentes escrevem em variáveis distintas, mas que coincidentemente estão na mesma linha de cache. O hardware não distingue posições lógicas, isto é, invalida a linha inteira a cada escrita, fazendo a linha "bater" entre os núcleos (cache line ping-pong). O resultado é uma penalidade severa de desempenho, mesmo sem condição de corrida lógica.

2.5. Geração de Números Aleatórios em Paralelo

2.5.1. rand()

A função rand() mantém um estado interno global compartilhado por todas as threads. Em ambientes paralelos, esse estado é geralmente protegido, fazendo com que as chamadas sejam efetivamente serializadas.

2.5.2. rand_r()

2.5.3. A função rand_r(unsigned int *seed) recebe uma seed como argumento, eliminando o estado global. Cada thread mantém sua própria semente local privada, permitindo geração de números totalmente independente e paralela.

2.6. Flag

```
gcc -o tarefa8 tarefa8.c -lm -fopenmp  
./tarefa8
```

2.7. RESOLUÇÃO EM CÓDIGO DA TAREFA

Segue no apêndice.

2.8. OUTPUT

Segue no apêndice.

3. CONCLUSÃO

As versões com `rand()` (v1 e v2) apresentaram desempenho inferior porque o gerador mantém um estado global compartilhado, serializando as threads e gerando tráfego constante de coerência de cache entre os núcleos. Já as versões com `rand_r()` (v3 e v4) eliminaram esse gargalo ao usar seeds privadas, permitindo geração de números verdadeiramente independente. A diferença entre v3 (0.074 s) e v4 (0.438 s) revela o efeito do falso compartilhamento: na v4, as posições do vetor `hits[]` residem nas mesmas linhas de cache, fazendo com que cada escrita de uma thread invalide a linha nos demais núcleos (protocolo MESI), provocando o cache line ping-pong. Já a v3, ao usar variáveis privadas, mantém o contador na cache local de cada núcleo, sem invalidações, e o critical final tem custo desprezível por ser executado apenas uma vez por thread.

Conclui-se que uma solução aparentemente mais paralela (sem critical) pode ser pior que uma com sincronização explícita, pois o falso compartilhamento, que é invisível no código mas severo no hardware, pode superar amplamente o custo de uma região crítica bem dimensionada.

4. APÊNDICE

4.1. RESOLUÇÃO

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <omp.h>

#define N 100000000

// no win não deu, então criando gerador de seed
static inline int meu_rand_r(unsigned int *seed) {
    *seed = (*seed) * 1103515245 + 12345;
    return (int)((*seed >> 16) & 0x7FFF);
}

#define MEU_RAND_MAX 32767

// VERSÃO 1: rand() + variável privada + critical
double estimar_pi_v1(Long n) {
    Long hits_global = 0;
    double inicio = omp_get_wtime();

    #pragma omp parallel
```

```

{
    long hits_local = 0;

    #pragma omp for
    for (long i = 0; i < n; i++) {
        double x = (double)rand() / RAND_MAX;
        double y = (double)rand() / RAND_MAX;
        if (x*x + y*y <= 1.0)
            hits_local++;
    }

    #pragma omp critical
    {
        hits_global += hits_local;
    }
}

double fim = omp_get_wtime();
printf("VERSAO 1: critical + rand() | tempo = %.4f s | pi ~ %.6f\n",
fim - inicio, 4.0 * hits_global / n);
return fim - inicio;
}

// VERSAO 2: rand() + vetor compartilhado
double estimar_pi_v2(long n) {
    int num_threads = omp_get_max_threads();
    long *hits = calloc(num_threads, sizeof(long));
    double inicio = omp_get_wtime();

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();

        #pragma omp for
        for (long i = 0; i < n; i++) {
            double x = (double)rand() / RAND_MAX;
            double y = (double)rand() / RAND_MAX;
            if (x*x + y*y <= 1.0)
                hits[tid]++; // falso compartilhamento aqui
        }
    }

    long hits_global = 0;
    for (int i = 0; i < num_threads; i++)
        hits_global += hits[i];

    double fim = omp_get_wtime();

```

```

    printf("VERSAO 2 vetor + rand() | tempo = %.4f s | pi ~ %.6f\n", fim -
início, 4.0 * hits_global / n);
    free(hits);
    return fim - início;
}

// VERSÃO 3: meu_rand_r() + variável privada + critical
double estimar_pi_v3(Long n) {
    Long hits_global = 0;
    double início = omp_get_wtime();

    #pragma omp parallel
    {
        Long hits_local = 0;
        // seed privada -> diferente em cada thread
        unsigned int seed = (unsigned int)time(NULL) ^
(omp_get_thread_num() * 7919);

        #pragma omp for
        for (Long i = 0; i < n; i++) {
            double x = (double)meu_rand_r(&seed) / MEU_RANDOM_MAX;
            double y = (double)meu_rand_r(&seed) / MEU_RANDOM_MAX;
            if (x*x + y*y <= 1.0)
                hits_local++;
        }

        #pragma omp critical
        {
            hits_global += hits_local;
        }
    }

    double fim = omp_get_wtime();
    printf("[v3] critical + rand_r() | tempo = %.4f s | pi ~ %.6f\n",
        fim - início, 4.0 * hits_global / n);
    return fim - início;
}

// VERSÃO 4: meu_rand_r() + vetor compartilhado
double estimar_pi_v4(Long n) {
    int num_threads = omp_get_max_threads();
    Long *hits = calloc(num_threads, sizeof(Long));
    double início = omp_get_wtime();

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        unsigned int seed = (unsigned int)time(NULL) ^ (tid * 7919);

```

```

        #pragma omp for
        for (long i = 0; i < n; i++) {
            double x = (double)meu_rand_r(&seed) / MEU_RANDOM_MAX;
            double y = (double)meu_rand_r(&seed) / MEU_RANDOM_MAX;
            if (x*x + y*y <= 1.0)
                hits[tid]++; // falso compartilhamento aqui
        }
    }

    long hits_global = 0;
    for (int i = 0; i < num_threads; i++)
        hits_global += hits[i];

    double fim = omp_get_wtime();
    printf("VERSAO 4 vetor + rand_r() | tempo = %.4f s | pi ~ %.6f\n",
        fim - inicio, 4.0 * hits_global / n);
    free(hits);
    return fim - inicio;
}

int main(void) {
    printf("N = %d pontos | threads = %d\n\n", N, omp_get_max_threads());

    double t1 = estimar_pi_v1(N);
    double t2 = estimar_pi_v2(N);
    double t3 = estimar_pi_v3(N);
    double t4 = estimar_pi_v4(N);

    printf("\nResumo dos tempos\n");
    printf("v1 (critical + rand()): %.4f s\n", t1);
    printf("v2 (vetor + rand()): %.4f s\n", t2);
    printf("v3 (critical + rand_r()): %.4f s\n", t3);
    printf("v4 (vetor + rand_r()): %.4f s\n", t4);

    printf("v1/v3 (impacto do rand X rand_r): %.2fx\n", t1/t3);
    printf("v4/v3 (impacto do falso compart.): %.2fx\n", t4/t3);

    return 0;
}

```

4.2. OUTPUT

```
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa8> gcc -o tarefa8 tarefa8.c -lm -fopenmp
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa8> ./tarefa8
N = 100000000 pontos | threads = 16

VERSAO 1: critical + rand() | tempo = 0.2410 s | pi ~ 3.141204
VERSAO 2 vetor + rand() | tempo = 0.7190 s | pi ~ 3.140807
[v3] critical + rand_r() | tempo = 0.0740 s | pi ~ 3.141248
VERSAO 4 vetor + rand_r() | tempo = 0.4380 s | pi ~ 3.141248

Resumo dos tempos
v1 (critical + rand()): 0.2410 s
v2 (vetor + rand()): 0.7190 s
v3 (critical + rand_r()): 0.0740 s
v4 (vetor + rand_r()): 0.4380 s
v1/v3 (impacto do rand X rand_r): 3.26x
v4/v3 (impacto do falso compart.): 5.92x
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa8> 
```